

C语言与程序设计

The C Programming Language

第2章 C语言的基本元素

王多强

1097412466

dqwang@hust.edu.cn

华中科技大学计算机学院



第二章第二部分：

2.5 运算符和表达式

2.6 类型转换

2.7 枚举类型

2.8 C99新增数据类型（自学）

2.5 运算符和表达式

- ◆ 表示运算功能的符号称为**运算符**。如： **+**、**-**、*****、**/**
- ◆ 被运算符操作的对象称为**操作数**。
- ◆ 由**运算符**和**操作数**组成的有意义的式子称为**表达式**。

如： $\text{sqrt}(b*b-4*a*c)$

$x = x * \text{PI} / 180$

表达式不是一句话，**后面不加分号**

$\text{fabs}(a_n) \geq \text{EPS}$

x



单个操作数（变量）也是表达式的一种形式

2.5.1 运算符概述

1、每个运算符都有特定的功能，如 **+**（加运算）、**-**（减运算）

2、运算符对操作数通常有**个数**和**类型**的要求

◆ **单目运算符**：一元运算，有1个操作数，如 **-b**

◆ **双目运算符**：二元运算，有2个操作数，如 **x + y**

◆ **三目运算符**：三元运算，有3个操作数，如 **x > y ? x : y**

条件运算符

➤ 运算符对操作数类型一般也有要求。

如：**%**的操作数只能是整数，不能为浮点数。

3. 运算符的优先级和结合性

- ◆ 当表达式中包括**多个运算符**时，将首先按“**优先级**”规则解释表达式的含义。（类似数学运算的**先乘除、后加减**）

如： $1+2*3$ 相当于 $1+(2*3)$

- ◆ 当一个操作数左、右两侧的运算符**优先级别相同**时，则按“**结合性**”规则来处理（即先做左侧的运算还是先做右侧的运算）。

如： $1+2-3$ 相当于 $(1+2)-3$

对表达式中运算符优先级和结合性的解释相当于给表达式**加括号**，使得表达式能够按照正确的**组合逻辑**执行计算。

C语言中共有45个运算符、15个运算优先级、2种结合性

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	括号的优先级：1 () [] -> .	圆括号 下标 间接引用结构体成员	左结合
2	! ~ ++、-- +、- (数据类型) &、* sizeof	逻辑非 按位取反 自增、自减 取正、取负 强制类型转换 取地址、间接引用 数据长度	右结合
3	*, /, %	乘、除、求余数	左结合
4	+, -	加、减	左结合
5	<<, >>	左移、右移	左结合
6	<, > >=, <=	大于、小于 大于等于、小于等于	左结合
7	=, !=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10		按位或	左结合
11	&&	逻辑与	左结合
12		逻辑或	左结合
13	?:	条件	右结合
14	=, +=, -=, *=, /=, %=, >>=, <<=, &=, ^=, =	赋值	右结合
15	,	逗号	左结合

优先级（从高到低）	运算符	运算符名称	结合性
括号的优先级：1	()	圆括号	左结合
	[]	下标	
	->	间接引用结构体成员	
	.		
	!	逻辑非	

优先级为1的运算符有：()、[]、->、.

[]：方括号，数组下标界定符

->：箭头
.
点号

结构和联合的成员选择符

()：圆括号，改变一个大表达式中子表达式的计算优先次序。

◆ 任何时候，只要有括号就先算括号内表达式的值；括号可以嵌套，对嵌套的括号，先处理内层括号再处理外层括号。

如： $x^*(y+z)$ $x^*((y+a)*z+b)$

4. 表达式的值和类型

表达式运算后都有一个运算结果。运算结果也是一个具有确定类型的值。这个值称为**表达式的值**，其类型称为**表达式值的类型**，简称为**表达式的类型**。

- ◆ 当不同类型的操作数进行“混合”运算时，结果数据（**表达式值**）的类型要按规则确定，这时候会**发生数据转换**，要特别注意。

如：**2+3** 结果为5， **整数**

2+3.0 结果为5.0， **浮点数**

类型的转换规则以后讲解

2.5.2 算术运算

- **算术运算**：加、减、乘、除、求余、正、负
- C语言的算术运算符：

符号	功能	操作数	优先级	结合性	示例	结果及说明
+	加法	二元运算	4	左结合	$3 + 6$	9
-	减法				$6 - 4$	2
*	乘法	二元运算	3	左结合	$3 * 8$	24
/	除法				$8 / 4$	2
%	求余				$7 \% 4$	3 ，即求两个整数相除后的余数
+	正值	一元运算	2	右结合	+3	3
-	负值				-5	-5

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	() [] -> .	圆括号 下标 间接引用结构体成员	左结合
2	! ~ ++、-- +、- (数据类型) &、* sizeof	逻辑非 按位取反 自增、自减 取正、取负 强制类型转换 取地址、间接引用 数据长度	右结合
3	*, /, %	乘、除、求余数	左结合
4	+, -	加、减	左结合
5	<<, >>	左移、右移	左结合
6	<, > >=, <=	大于、小于 大于等于、小于等于	左结合
7	==, !=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10		按位或	左结合
11	&&	逻辑与	左结合
12		逻辑或	左结合
13	?:	条件	右结合
14	=, +=, -=, *=, /=, %=, >>=, <<=, &=, ^=, =	赋值	右结合
15	,	逗号	左结合

单目运算符

说明事项:

- (1) 注意C语言运算符和纯数学符号的差异, 如乘法符号: * 和 ×
- (2) 同一个符号, 在上下文中表示的运算可能不同, 如: +、-



- (3) 先乘除、后加减: $3+4*5$ 相当于 $3+(4*5)$ 乘除优先级为3
加减优先级为4

- (4) 加、减、乘、除都是左结合, 正、负是右结合:

$$3+4-5 \text{ 相当于 } (3+4)-5$$

- (5) 正、负是单目运算符, 优先级为2:

$$3--4*5 \text{ 相当于 } 3-((-4)*5)$$

(6) 对于 $+$ 、 $-$ 、 $*$ 、 $/$ (加、减、乘、除)

- ◆ 当两个操作数都是**整数**的时候，运算结果也是**整数**。
- ◆ 有一个操作数是**浮点数**时，运算结果就是**浮点数**。

如：

$$1+1=2$$

$$1.0+1=2.0$$

$$1-1.0=0.0$$

$$2*5=10$$

$$2*5.0=10.0$$

$$5/2=2$$

$$1/2=0$$

$$5.0/2=2.5$$

$$1.0/2=0.5$$

整数除法是**向下取整**

(7) $\%$ 的两个操作数都必须为整数

$$5\%2 \Rightarrow 1 \quad \checkmark$$

$$5.0\%2 \text{ 或 } 5\%2.0 \quad \times$$

例2.7 求出所有的水仙花数

“水仙花数” 是一个三位数，其**各位数字的立方和**等于**该数本身**。

如，**153**: $153 = 1^3 + 5^3 + 3^3$

解题思路——解决以下几个问题即可：

- ◆ 怎样从一个三位数中分离出百位数、十位数和个位数？

设**x**为一个三位数， (1) 求x的百位数: $i = x / 100$

(2) 求x的十位数: $j = (x - i * 100) / 10$

(3) 求x的个位数: $k = x \% 10$

- ◆ 求一个数的立方: $y * y * y$

- ◆ 求三个数的立方和: $i * i * i + j * j * j + k * k * k$

- ◆ 还需要一个**循环**: 对**100~999**进行判别，找出其中水仙花数。

```
#include <stdio.h>

int main(void)
{
    int x, i, j, k;          /* 定义变量, x代表三位数, i, j, k分别是x的三个数字 */
    for (x=100; x<=999; x=x+1) /* 从100开始循环到999 */
    {
        i = x/100;           /* 提取百位数 */
        j = (x-i*100)/10;    /* 提取十位数 */
        k = x%10;            /* 提取个位数 */
        if (x == i*i*i+j*j*j+k*k*k) /* 测试是否是水仙花数 */
            printf("%5d", x); /* 如果是水仙花数, 就输出 */
    }
    return 0;
}
```

程序运行后输出4个水仙花数:

153、370、371、407

2.5.3 关系运算

关系运算用于**比较两个数的大小**。如果两个操作数之间的关系成立，则运算结果为**真**，否则结果为**假**。

但C语言的基本数据类型中没有专门的逻辑类型，而是**用数值1代表逻辑真**，**数值0代表逻辑假**，所以**C语言中关系运算的结果就是1或者0**，且为**int型**。

注：C99中扩展了基本数据类型，引入了正式的布尔类型_bool，并定义符号常量true和false分别代表逻辑真和逻辑假，但值依然是真为1、假为0。（自学一下）

C语言有6个关系运算符：

符号	功能	操作数	优先级	结合性	示例	结果及说明
>	大于	二元运算	6	左结合	5>3	1
>=	大于等于				5>=3	1
<	小于				5<3	0
<=	小于等于				5<3	0
==	等于		7		5==3	0
!=	不等于				5!=3	1

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	() [] -> .	圆括号 下标 间接引用结构体成员	左结合
2	! ~ ++、-- +、- (数据类型) &、* sizeof	逻辑非 按位取反 自增、自减 取正、取负 强制类型转换 取地址、间接引用 数据长度	右结合
3	*, /, %	乘、除、求余数	左结合
4	+, -	加、减	左结合
5	<<, >>	左移、右移	左结合
6	<, > >=, <=	大于、小于 大于等于、小于等于	左结合
7	=, !=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10		按位或	左结合
11	&&	逻辑与	左结合
12		逻辑或	左结合
13	?:	条件	右结合
14	=, +=, -=, *=, /=, %=, >>=, <<=, &=, ^=, =	赋值	右结合
15	,	逗号	左结合

关系运算举例

字母	ASCII码
A	65
a	97
b	98

根据变量说明，给出下列表达式的值。

```
int x=4, y=3, z=2;
```

```
char c='a';
```

(1) $c == \underline{'A'} + 32$ **1**

(2) $\underline{c} + 1 \neq 'b'$ **0**

(3) $\underline{x - y} \leq 10$ **1**

(4) $z = \underline{x > y}$ **1**

(5) $x > y > z$ **0** $\longrightarrow$ **$(x > y) > z$: $1 > 2$**

(6) $x > y < z$ **1** $\longrightarrow$ **$(x > y) < z$: $1 < 2$**

注：

- ◆ 字符量在表达式中用其ASCII码值参与运算
- ◆ 利用**优先级**和**结合性**解释表达式计算过程
- ◆ **算术运算 > 关系运算 > 赋值运算**

根据关系运算的**左结合性**，等价于：

注意C语言和数学的差异

(1) 数学上判断**x是否在区间(a, b)**中时，习惯写成:

$$a < x < b$$

但C程序中表达式 “ $a < x < b$ ” 的含义与数学表示的含义不同:

C语言中, $a < x < b$ 仅等价于 $(a < x) < b$

因此判断 $x \in (a, b)$? , 应写成: $a < x \ \&\& \ x < b$

&&: 逻辑运算符, 表示 **与、并且**

即: $a < x$ 并且 $x < b$

如果是区间 $[a, b]$, 则要写成 $a \leq x \ \&\& \ x \leq b$

(2) 数学中表示**相等**一般用一个等号，但C语言表示相等要用**==**。

- ◆ C语言中的单个等号**=**是**赋值运算**。
- ◆ 误用的话可能会因为运行时的逻辑错误而导致不正确的结果——程序员很容易犯的错误！

例如： **if (grade == 'A')**

```
printf("Very Good! ");
```

```
/*当成绩的等级为A等时，输出Very Good!*/
```

而： **if (grade = 'A')**

```
printf("Very Good!");
```

```
/*不论事先成绩的等级是几等，执行时总输出Very Good!*/
```

思考为什么？

2.5.4 逻辑运算

◆ **逻辑运算**表示对操作对象的**与、或、非**逻辑关系。

◆ 有3个逻辑运算符：

!	逻辑非	单目运算	优先级 2
&&	逻辑与	双目运算	优先级 11
 	逻辑或	双目运算	优先级 12

表达式形式：**e1&&e2** 或 **e1||e2**

e1	e2	e1&&e2	e1 e2
0	0	0	0
0	非0	0	1
非0	0	0	1
非0	非0	1	1

&& 和 || 的真值表

表达式形式：**!e**

e	!e
0	1
非0	0

! 的真值表

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	()	圆括号	左结合
	[]	下标	
	->	间接引用结构体成员	
	.		
2	!	逻辑非	右结合
	~	按位取反	
	++、--	自增、自减	
	+, -	取正、取负	
	(数据类型)	强制类型转换	
	&、*	取地址、间接引用	
	sizeof	数据长度	
3	*, /, %	乘、除、求余数	左结合
4	+, -	加、减	左结合
5	<<, >>	左移、右移	左结合
6	<, >	大于、小于	左结合
	>=, <=	大于等于、小于等于	
7	==, !=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10		按位或	左结合
11	&&	逻辑与	左结合
12		逻辑或	左结合
13	?:	条件	右结合
14	=, +=, -=, *=, /=, %=, >>=, <<=, &=, ^=, =	赋值	右结合
15	,	逗号	左结合

单目运算符

- ◆ 逻辑运算的操作数可以是**0**和**非0**的任何数值，其中

0：表示false **非0**：表示true

- ◆ 逻辑表达式结果的值只是：**1** (真) 或 **0** (假)，int型

如：表达式 值

5&&0 **0**

100||0 **1**

!101 **0**

!0 **1**

!1 **0**

!'a' **0**

```
int y = !!101;
```

◆ 可以用逻辑运算符将其它表达式连接形成一些复杂的判定表达式

(1) 判断整型变量a的值是否是偶数，是则结果为1，否则为0

!(a%2) 或 a%2==0 【例2.12】

(2) 判断字符变量c是否是英文字母

(c>='a' && c<='z') || (c>='A' && c<='Z') 【例2.13】

等价于 **c>='a' && c<='z' || c>='A' && c<='Z'**

(3) 判断某一年year是闰年还是平年：

!(year%4) && year%100 || !(year%400) 【例2.14】

闰年的判别条件：如果某一年份能被4整除但不能被100整除，或者能被400整除，那么该年份是闰年。

程序运行时，在处理含有**&&**、**||** 表达式时往往采用**短路法**
则优化方式以提高速度：

- **e1&&e2**：一旦发现**e1等于0**，就不再计算 e2，**表达式的值等于0**
- **e1 || e2**：一旦发现**e1等于1**，就不再计算 e2，**表达式的值等于1**

例如， **x >= 0.0 && sqrt(x) <= 7.7**

若**x=-1**，则**结果为0**，并且不会报错

若写成： **sqrt(x) <= 7.7 && x >= 0.0** 结果是什么？

若x=-1，则报错，因为sqrt不能对负数求平方根

2.5.5 自增和自减运算

++ : **自增**, **作用于变量**, 如: ++x, 相当于 $x = x + 1$

即使变量自身的值加1

-- : **自减**, **作用于变量**, 如: --x, 相当于 $x = x - 1$

即使变量自身的值减1

◆ ++、--是单目运算符, 优先级2, 右结合

所有运算符的优先级和结合性规则

优先级 (从高到低)	运算符	运算符名称	结合性
1	()	圆括号	左结合
	[]	下标	
	->	间接引用结构体成员	
2	!	逻辑非	右结合
	~	按位取反	
	++、--	自增、自减	
	+、-	取正、取负	
	(数据类型)	强制类型转换	
单目运算符	&、*	取地址、间接引用	
	sizeof	数据长度	

◆ ++和--有**前缀式**和**后缀式**两种形式：

前缀式：把++或--放在变量的前面，如：**++x, --x**

后缀式：把++或--放在变量的后面，如：**x++, x--**

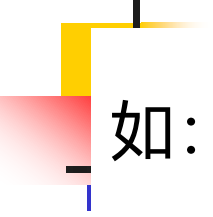
■ 前缀式与后缀式的区别

前缀式：先对操作数执行++或--运算，使其值加1或减1，然后再使用**操作数的新值**参与后续运算

后缀式：先用**操作数的旧值**参与别的运算，然后再对它作++或--运算，使其值加1或减1（**新值不影响之前做的其它运算**）。

有什么影响呢？

对变量自身而言，不管前缀式还是后缀式结果是一样的，都使其值加1或减1，但对它们参与的表达式而言会存在不同。



如：设 `int x = 10, y;` 观察以下表达式和变量的值。

`y = ++x;` 结果：y等于11, x等于11

`y = --x;` 结果：y等于9, x等于9

`y = x++;` 结果：y等于10, x等于11

`y = x--;` 结果：y等于10, x等于9

注意事项:

- 1) 自增自减运算要改变操作数的值, 所以只能作用于**变量**上, 不能作用在常量上, 也不能作用的表达式上。

如: $5++$ **×** $(a+b)--$ **×**

表达式不是一个实体, 没有存储空间, 不能“自增”

- 2) 单目运算符的优先级都是**2级**, 结合性为**右结合**。

如: $-i++$ 相当于 $-(i++)$

如果开始 $i=3$, 则**表达式结果为 -3**, i 为4

如: $y=-i++$; 则 y 等于-3

自增自减有很多灵活的组合, 经常考!

例2.16 计算 $1+2+3+\dots+n$, n 从键盘输入

算法思想: 输入 n , 然后对 $1\sim n$ 之间的整数累加求和, **用循环实现**。

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int n, i, sum;
```

```
    scanf("%d", &n); /* 输入n */
```

```
    i = 1; /* i 称为循环变量, 用于控制循环, 这里初始化为1 */
```

```
    sum = 0; /* sum是累加器, 初始化为0 */
```

```
    while (i<= n) /* while循环, 当i<=n时, 反复将i加到sum上 */
```

```
        sum = sum + i++; /* 先将i的旧值加到sum, 然后i自增, 进入下一次循环 */
```

```
    printf("sum=%d\n", sum); /* 输出sum的值 */
```

```
    return 0;
```

```
}
```

思考: 如果求和语句改成

sum = sum + ++i;

程序其它部分要做什么改变?

i=0;

while(i<n)

◆ 相比前缀式，对**后缀式**的理解有点复杂，如：

$$\text{sum} = \text{sum} + \text{i}++;$$

先用**变量i的旧值**参与求和，然后再改变变量i的值。可见，变量值的更改不是马上发生，而是**发生在对其原值引用之后的某个时刻**——这种现象称为**后缀式的计算延迟**。

那么要延迟到什么时候才执行对变量的加1或减1操作呢？

对此，C语言并没有强制要求引用完旧值就马上更新，而是定义了**序列点**的概念。

序列点：是一个程序执行中的时刻点，C语言规定，在这个点之前所有**对变量值的改变**必须完成。

因此**只要在此之前完成更新即可，未必立刻马上！**但“在此之前”是一个宽泛的概念，有可能是“在此之前”的任何时刻。

C标准中定义的序列点有以下两大类情况：

(1) **&&、||、?: 或 , :** 在这些运算符的**第一个操作数**之后。

(2) **完整表达式结束**：包括表达式语句末尾的分号处、return 语句中的表达式末尾的分号处、if、switch、do、while、for语句的表达式右括号处，for语句中控制语句的两个分号处。

例2.17 后缀式++(或--)表达式举例

设 `int a=1, b=0`; 计算下面表达式的值

1) `a-- && (a+b)`

分析： `&&` 的优先级低，所以要先算 `a--`、`a+b`，最后再 `&&`。

问题： 先计算 `a--`，再计算 `a+b`，还是先计算 `a+b`，再计算 `a--`？

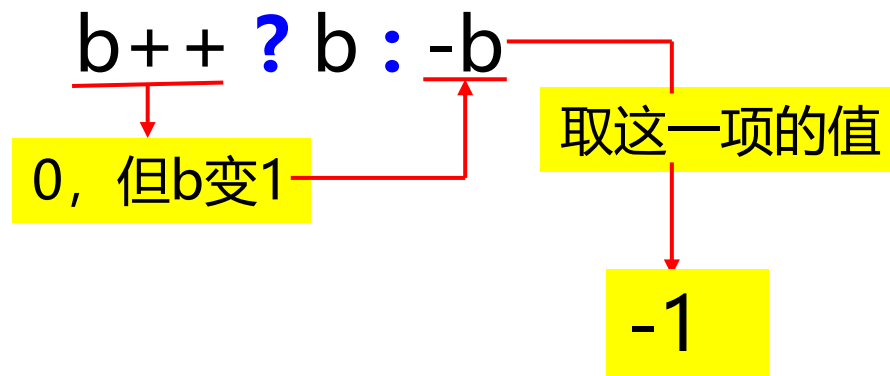
对一般的二元运算，C语言没有明确规定**左、右子表达式操作数**的计算顺序，由编译器自定决定。但对 **`&&`**、**`||`**、**`,`** 三个运算，C语言明确规定了计算顺序：**从左向右**。所以上述表达式是**先计算 `a--`，再计算 `a+b`**。又根据序列点的定义，在 `&&` 之前要完成 `a` 的更新，所以最后的结果是：**`1&&(0+0) = 1&&0 = 0`**。

设 `int a=1,b=0`; 计算下面表达式的值

2) `b++ ? b : -b`

`e1?e2:e3` : 条件运算符, 如果e1为真, 取e2, 否则取e3。

根据序列点的定义, 序列点在 ? 之前, 所以结果是:



所以上述表达式值为-1, b的值为1。

设 `int a=1,b=0;` 计算下面表达式的值

3) `b+++b++`

特殊之处：根据序列点的定义，该表达式的序列点应该在表达式**最后结束之处**，中间没有其它序列点。但如果认为**在最后才**发起更新操作，则该式的值等于0。

实际情况：`int c= b+++b++;`

`printf("c=%d, b=%d", c, b);`

输出将是：`c=1, b=2`

最后b等于2是确定的

为什么？—— 还是序列点的问题！

原因分析：

C语言规定，**变量更新在序列点之前完成即可**。这种情况下，

- ◆ 如果在最后时刻才做， $b++ + b++$ 的结果就是0。
- ◆ 而如果在之前的其它时刻做，就有不同的结果。从实际结果看，是**每次 $b++$ 后都更新了b的值**。

对还是错？

结果是0或者1都不算错，这是**编译器的行为**，不是C语言的硬性要求，所以编程之前要了解一下所用编译器的特性。

设 `int a=1,b=0`; 计算下面表达式的值

4) `b+++ (b+a)`

计算方向的影响：如前所述，加法运算的左右操作数计算顺序可以从左向右，也可以从右向左。

从左向右：表达式结果为1或者2（看编译器对序列点的处理）

从右向左：表达式结果为1

实测：`int c= b+++ (b+a);`

`printf("c=%d, b=%d",c b);`

输出将是：`c=2, b=1`

b等于1是确定的

说明计算顺序是从左向右，并在**`b++`**后对**`b`**立刻做了更新。

例2.15 统计输入正文的字符数和行数

设计思路：

(1) 用C语言的标准库函数**getchar()**接收从键盘输入的字符：

```
c = getchar( );
```

```
/* c是字符变量, getchar()输入的字符存到c中 */
```

用**循环的方式**，一次一个字符、连续输入。

(2) **统计字符数**：每输入一个字符，**计数器nc加1**（nc是预先定义好的整型变量，初值为0，不断加1）。

(3) **统计行数**：每读到一个换行符（'\n'）就表示输入了一行，所以统计换行符的个数即可，**用计数器nl统计行数**（nl也是预先定义好的整型变量，初值为0，不断加1）。

(4) 什么时候输入结束?

- ◆ **getchar()**遇到**文件结束标志EOF**时结束循环，然后输出nc和nl的值即可。

- ◆ **文件结束标志怎么输入?**

按下**Ctrl+Z** (DOS系统) 或**Ctrl+D** (UNIX系统) 即可。

注: **EOF是系统预定好的一个符号常量，值等于-1。**

```
#define EOF -1
```

按下Ctrl+Z (DOS系统) 可以输入这个值给getchar()。

```
#include <stdio.h>
```

```
#define EOF -1
```

```
int main(void)
```

```
{
```

```
    int c, nc, nl;
```

```
    printf("Input a text end of ctrl+z:\n"); /* 提示语 */
```

```
    nc = nl = 0;
```

```
    while ((c=getchar()) != EOF)
```

```
    {
```

```
        ++nc;                                /* 字符数自增1, 可以用nc++代替 */
```

```
        if (c == '\n')
```

```
            ++nl;                            /* 行数自增1, 可以用nl++代替 */
```

```
    }
```

```
    printf("nc=%d,nl=%d\n", nc, nl);
```

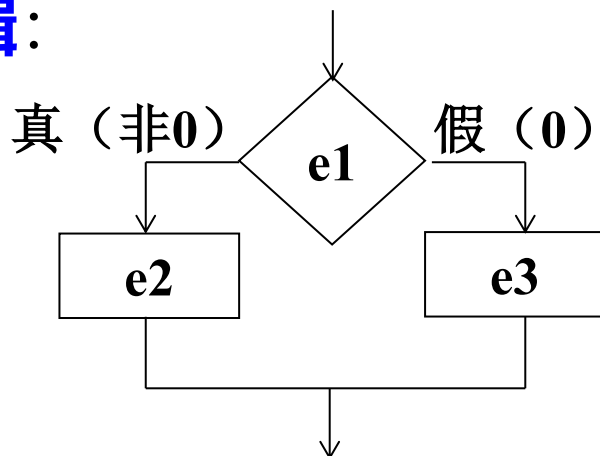
```
    return 0;
```

```
}
```


2.5.6 条件运算 ? :

- ◆ **条件运算符**是C语言中唯一的一个三目运算符。
- ◆ 条件运算表达式的一般形式： **$e1 ? e2 : e3$**

执行逻辑：



例： **$x=10;$**

$y=x>9? 100: 200$

/ 结果 $y=100$ */*

- ◆ **优先级13，右结合（先右后左）**

例： **$x = a > 0 ? 1 : a < 0 ? -1 : 0$**

等价于 **$x = a > 0 ? 1 : (a < 0 ? -1 : 0)$**

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	() [] -> .	圆括号 下标 间接引用结构体成员	左结合
2	! ~ ++、-- +、- (数据类型) &、* sizeof	逻辑非 按位取反 自增、自减 取正、取负 强制类型转换 取地址、间接引用 数据长度	右结合
3	*, /, %	乘、除、求余数	左结合
4	+, -	加、减	左结合
5	<<, >>	左移、右移	左结合
6	<, > >=, <=	大于、小于 大于等于、小于等于	左结合
7	=, !=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10		按位或	左结合
11	&&	逻辑与	左结合
12		逻辑或	左结合
13	? :	条件	右结合
14	=, +=, -=, *=, /=, %=, >>=, <<=, &=, ^=, =	赋值	右结合
15	,	逗号	左结合

三目运算符

2.5.7 赋值运算

赋值运算的基本功能：将数据存入指定的存储单元。

1、赋值表达式的基本形式： **<变量> = <子表达式>**

- ◆ **=**：赋值运算符
- ◆ 含义：将右侧子表达式的**值**赋给左侧的**变量**
- ◆ 赋值表达式的值和类型：等于左操作数的值和类型

赋值表达式的**左操作数**必须是**左值** (lvalue) ，即**有用以保存数据的实际内存字节空间**的量。

- 变量可以作为左值，间接访问指针所得的对象也可以。
- 表达式不能作为左值，如：**a + b = 5**。 ❌
- 常量也不能作为左值，如：**5 = 6**。 ❌

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	() [] -> .	圆括号 下标 间接引用结构体成员	左结合
2	! ~ ++、-- +、- (数据类型) &、* sizeof	逻辑非 按位取反 自增、自减 取正、取负 强制类型转换 取地址、间接引用 数据长度	右结合
3	*/、%	乘、除、求余数	左结合
4	+、-	加、减	左结合
5	<<、>>	左移、右移	左结合

- 赋值运算符的优先级是14，结合性是右结合性

如: $a=b=c=3+x;$ 等价于 $a=(b=(c=(3+x)))$;

11	&&	逻辑与	左结合
12		逻辑或	左结合
13	? :	条件	右结合
14	=, +=, -=, *=, /=, %=, >>=, <<=, &=, ^=, =	赋值	右结合
15	,	逗号	左结合

注：

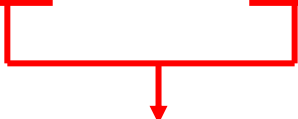
C语言把**赋值**处理为运算符，进而得到**赋值表达式**（<变量> = <子表达式>），赋值表达式可以当作一个整体来使用。

赋值表达式的值和类型等于左操作数的值和类型。 如

a=2 a的值等于2，整型；整个赋值表达式的值也是2，整型

b=3

若有 **x=a+b**，可以写成：

$$x = \underbrace{(a=2) + (b=3)}$$


先计算赋值表达式的值，然后用该值参与其它运算

2、复合赋值运算

复合赋值运算： 赋值运算符 = 和另一双目运算符的组合

如： $+=$, $-=$, $*=$, $/=$, $\%=$

- ◆ 大多数双目运算符都可以和 “=” 一起构成复合赋值运算符。
- ◆ 含义： $x \text{ op} = y$ 等价于 $x = x \text{ op} y$

如： $i += 2$ 等价于 $i = i + 2$

$y /= x + 10$ 等价于 $y = y / (x + 10)$

$x * = k = m + 5$ 等价于 $x = x * (k = m + 5)$

右侧子表达式作为一个整体对待，右结合

特例： $s[i++] += 1$ 和 $s[i++] = s[i++] + 1$ 不等价！

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	() [] -> .	圆括号 下标 间接引用结构体成员	左结合
2	! ~ ++、-- +、- (数据类型) &、* sizeof	逻辑非 按位取反 自增、自减 取正、取负 强制类型转换 取地址、间接引用 数据长度	右结合
3	*/、%	乘、除、求余数	左结合
4	+、-	加、减	左结合

- 复合赋值运算符和简单赋值运算符一样，优先级是14，结合性是右结合性

11	&&	逻辑与	左结合
12		逻辑或	左结合
13	? :	条件	右结合
14	=、+=、-=、*=、/=、%=、>>=、<<=、&=、^=、 =	赋值	右结合
15	,	逗号	左结合

2.5.8 逗号运算

C语言中，**逗号**也是一种运算符：

- ◆ 逗号运算符： **,**

- ◆ 逗号运算表达式的一般形式： **$e_1, e_2, \dots, e_n$**

其中每个 e_i 是其它形式的子表达式；

- ◆ **左结合**，从左往右计算：**先计算 e_1 ，再计算 e_2 ，...**

- ◆ **逗号表达式的值和类型**：与**最后 e_n** 的值和类型相同

例： (1) **$x=(i=4, i\%3)$** 表达式的值为1， x为1
 (2) **$x=i=4, i\%3$** 表达式的值为1， x为4



◆ 逗号表达式可以看作是几个子表达式用 ‘,’ 连结而成的一个新表达式

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	() [] -> .	圆括号 下标 间接引用结构体成员	左结合
2	! ~ ++、-- +、- (数据类型) &、* sizeof	逻辑非 按位取反 自增、自减 取正、取负 强制类型转换 取地址、间接引用 数据长度	右结合
3	*/、%	乘、除、求余数	左结合

逗号运算符的优先级最低。

在表达式中有逗号运算符时，总是先处理其它的子项，最后处理逗号运算符。

12		逻辑或	左结合
13	? :	条件	右结合
14	=、+=、-=、*=、/=、%=、>>=、<<=、&=、^=、 =	赋值	右结合
15	,	逗号	左结合

注意:

并不是任何地方出现的逗号都是**逗号运算符**，只有**作为语句出现的逗号**才构成逗号运算表达式。

例如：**函数参数**也是用逗号分隔的，但这里的逗号仅是一般的分隔符，不能视为逗号运算符，参数表也不是逗号表达式。

```
printf("%d, %d", 2, 3);
```

对比: `printf("%d,%d", (2, 3), 3 );` `/* 输出: 3, 3 */`



逗号表达式

例2.19 输入一串数字字符，将其转换为一个十进制整数赋值给x。

分析：

◆ 输入数字串：用**getchar() + 循环**的方式连续输入多个数字字符并组合在一起就构成一个数字字符串。

◆ 数字串怎么转换成整数？（十进制）

一个数字字符：**数字字符-'0'**就是它对应的数值。

如：**0** == '0' - '0'，**1** == '1' - '0' 等。

“多位”数字串：先把**每位的数码符**分别转换成对应数值，
然后根据进位制**乘基求和**即可。

如：**123** == (**1*10** + **2**)***10** + **3**

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    char c;    /* 字符变量c存放输入的字符 */
```

```
    int x;     /* 整型变量x存放转换的整数 */
```

```
    x=0
```

判断c是否是数字字符，
遇非数字字符就结束循环

```
    for ( c=getchar(); c>='0'&& c<='9'; c=getchar())
```

```
        x = 10 * x + c - '0';    /* c-'0': 将数字字符转换成对应的整数 */
```

```
    printf("x=%d\n", x);
```

```
    return 0;
```

```
}
```



123456 = (((((1*10 + 2)*10 + 3)*10 + 4)*10 + 5)*10 + 6

2.5.9 sizeof运算

sizeof：单目运算符，求**数据类型的存储长度**，即存储该数据类型的一个**数据对象**所需的字节数。

◆ 有两种使用形式：

(1) **sizeof (类型名)**：直接求出指定数据类型的存储长度。

如：sizeof(char) 等于1 sizeof(int) 等于2

(2) **sizeof (表达式)**：先识别表达式结果的类型，然后求该类型的存储长度。

如：double x; **sizeof x** 等于8



不引起歧义的情况下，括号可以省略，但要加空格隔开

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	() [] -> .	圆括号 下标 间接引用结构体成员	左结合
2	! ~ ++、-- +、- (数据类型) &、*	逻辑非 按位取反 自增、自减 取正、取负 强制类型转换 取地址、间接引用	右结合
	sizeof	数据长度	
3	*/、/、%	乘、除、求余数	左结合
4	+、-	加、减	左结合
5	sizeof是优先级很高的单目运算符		左结合
6	>=、<=	大于等于、小于等于	左结合
7	=、!=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10		按位或	左结合
11	&&	逻辑与	左结合
12		逻辑或	左结合
13	? :	条件	右结合
14	=、+=、-=、*=、/=、%=、>>=、<<=、&=、^=、 =	赋值	右结合
15	,	逗号	左结合

sizeof举例

假设int类型占用2字节，则：

sizeof(long) /* 值为4 */

double x; sizeof x /* 值为8 */

int c[10]; sizeof (c) /* 值为20 */

数组可以看作一种扩展类型

c是一个有10个元素的整型数组，每个元素占2个字节，所以共占20个字节

int a=10,b=10; sizeof (a+b) /* 值为2*/

虽然a+b的值是20，但该值是整型数，sizeof (a+b)仅是求表达式**结果的类型**的长度，所以sizeof (a+b)的值是2.

分析：如果是 **sizeof a+b**，则表达式的值是多少？

对于 **sizeof(表达式)** 的形式，没有必要先求表达式本身的值再求 **sizeof(表达式)** 的值。而是只要知道**表达式的类型**就可以确定**sizeof(表达式)**的值，所以编译器将sizeof(•)放在编译分析时直接做：**根据表达式的成分就可分析出表达式的类型，然后就可确定sizeof(•)的值，而在程序运行态时不再对表达式进行运算**——程序里把**sizeof**视为一个**常量表达式**（**类型长度是固定的**）。

例如：设有程序段

```
short x=1;
```

```
sizeof(++x);
```

```
printf("x=%d\n", x);    /*问：输出是什么？*/
```

编译时这条语句就被处理了：仅根据++x的类型就计算了sizeof(++x)的值，没实际做++x运算，所以x的值没变。而在运行时也不再执行这条语句，x也就没递增，值仍为1。

输出：1

2.5.10 位运算符和位运算表达式

- **字节**是C语言最小的存取单位，但最小的数据单位是**位**。
- **位运算**是施加在**位**上的运算。C语言有6个位运算符：

符号	功能	操作数	优先级	结合性	示例	结果及说明
~	按位求反	一元运算	2	右结合	~-5	?
&	按位与	二元运算	8	左结合	5&3	?
^	按位异或		9		5^3	?
	按位或		10		5 3	?
<<	左移		5		5<<3	?
>>	右移				5>>3	?

所有运算符的优先级和结合性规则

优先级(从高到低)	运算符	运算符名称	结合性
1	() [] -> .	圆括号 下标 间接引用结构体成员	左结合
2	!	逻辑非	右结合
	~	按位取反	
2	++、-- +、- (数据类型) &、* sizeof	自增、自减 取正、取负 强制类型转换 取地址、间接引用 数据长度	右结合
3	*/、/、%	乘、除、求余数	左结合
4	+、-	加、减	左结合
5	<<、>>	左移、右移	左结合
6	<、> >=、<=	大于、小于 大于等于、小于等于	左结合
7	=、!=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10		按位或	左结合
11	&&	逻辑与	左结合
12		逻辑或	左结合
13	? :	条件	右结合
14	=、+=、-=、*=、/=、%=、>>=、 <<=、&=、^=、 =	赋值	右结合
15	,	逗号	左结合

单目运算符

1、按位求反（~）

一位二进制数的按位取反：**0变1、1变0**

按位取反的表达式形式：**~ 操作数**

操作：对操作数二进制表示的每个位**逐位取反**。

如：设整数用2字节表示，求 **~-5**

~ 11111111111111011



00000000000000100

5₁₀的二进制表示：

0000000000000101₂

注：虽然位运算能作用在“位”上，但操作数是**以字节为单位**组织的，没有独立的“位”数据，位运算施加在所有位上。

2、按位与 (&)

一位二进制数的按位与： $0 \& 0 = 0$

$0 \& 1 = 0$

$1 \& 0 = 0$

$1 \& 1 = 1$

全1为1，否则为0

按位与的表达式形式：操作数1 & 操作数2

操作：对表达式左、右操作数二进制表示的每个位逐位“与”

如：设整数用2字节表示，求 $5 \& 3$

000000000000000101

& 000000000000000011

000000000000000001

5_{10} 的二进制表示

3_{10} 的二进制表示

1_{10} 的二进制表示

例 2.20 判断一个整数是否为偶数： $!(x \& 1)$

这里没有进位、借位的概念

3、按位或（|）

一位二进制数的按位或： $0|0=0$

$$0|1=1$$

$$1|0=1$$

$$1|1=1$$

有1即为1，否则为0

按位或的表达式形式： **操作数1 | 操作数2**

操作：对表达式左、右操作数二进制表示的每个位逐位“或”

如：设整数用2字节表示，求 $5|3$

000000000000000101

5_{10} 的二进制表示

| 000000000000000011

3_{10} 的二进制表示

000000000000000111

7_{10} 的二进制表示

这里没有进位、借位的概念

4、按位异或 ($\wedge$)

一位二进制数的按位异或： $0 \wedge 0 = 0$

$$0 \wedge 1 = 1$$

$$1 \wedge 0 = 1$$

$$1 \wedge 1 = 0$$

相同为0，相异为1

也相当于没有进位的加法

按位异或的表达式形式：**操作数1** $\wedge$ **操作数2**

操作：对表达式左、右操作数二进制表示的每个位**逐位**“异或”

如：设整数用2字节表示，求 $5 \wedge 3$

000000000000000101

5_{10} 的二进制表示

$\wedge$ 000000000000000011

3_{10} 的二进制表示

000000000000000110

6_{10} 的二进制表示

与0异或不变，与1异或变反

这里没有进位、借位的概念

掩码（模、逻辑尺）(mask)

掩码：一个每位具有指定0/1的二进制数，也称为模、逻辑尺。

如：1111111100000000₂ 0xff00

◆ 利用与掩码的按位与、或、异或运算，可以将一个数（二进制表示）的特定位置0、置1或取反。

如：设整数用2字节表示，int x=0x68d1₁₆ (0110100011010001₂)

x&mask

0110100011010001

& 1111111100000000

0110100000000000

将x的低字节置为0，高字节保留

0x6800

x|mask

0110100011010001

| 1111111100000000

1111111111010001

将x的低字节保留，高字节置为1

0xffd1

x^mask

0110100011010001

^ 1111111100000000

1001011111010001

将x的低字节保留，高字节翻转

0x97d1

5、左移 (<<)

左移表达式的形式： $e \ll n$

操作：将左操作数e的值（二进制表示）向左移n位（n为整数），
低n位填0，高位移出的n位舍弃。如，**以一字节数据为例**

$$(00001011)_2 \ll 2 = (00101100)_2$$

$$9_{10} \longrightarrow 36_{10}$$

注：左移1位相当于该数的值乘以2。

如上式, 9左移2位, 相当于乘4, 即 $9*4=36$ 。

6、右移 (>>)

右移表达式形式: $e >> n$

操作: 将左操作数e的值 (二进制表示) 向右移n位 (n为整数) ,

低位移出的n位舍弃, 而**高n位分情况讨论**:

- ◆ 若e是无符号类型数据, 则高n位填入0, 如:

$$\begin{array}{ccc} \text{高2位} & & \text{高2位填0} \\ \underline{(00001011)}_2 >> 2 = \underline{(00000010)}_2 \end{array}$$

- ◆ 若e是有符号类型数据, 一些机器填0 (称为**逻辑移位**) , 而另一些机器填符号位 (称为**算术移位**) , 如:

$$\begin{array}{ccc} \text{高2位} & & \text{高2位填0} \\ \bullet \text{ 逻辑移位: } \underline{(11110101)}_2 >> 2 = \underline{(00111101)}_2 \end{array}$$

$$\begin{array}{ccc} \text{符号位} & & \text{高2位填符号位} \\ \bullet \text{ 算术移位: } \begin{array}{c} \uparrow \\ (11110101)_2 \end{array} >> 2 = \underline{(11111101)}_2 \end{array}$$

以
一
字
节
数
据
为
例

以一字节数据为例

无符号数右移： $(00001011)_2 \gg 2 = (00000010)_2$
 $9 \longrightarrow 2$

有符号逻辑右移： $(11110101)_2 \gg 2 = (00111101)_2$
 $-9 \longrightarrow 61$

有符号算术右移： $(11110101)_2 \gg 2 = (11111101)_2$
 $-9 \longrightarrow -2$

有符号整数算术右移、无符号整数右移（可视为正整数算术右移），
每右移1位相当于该数除一次2。

如上式，9左移2位，相当于除以4，即 $9/4=2$ ，

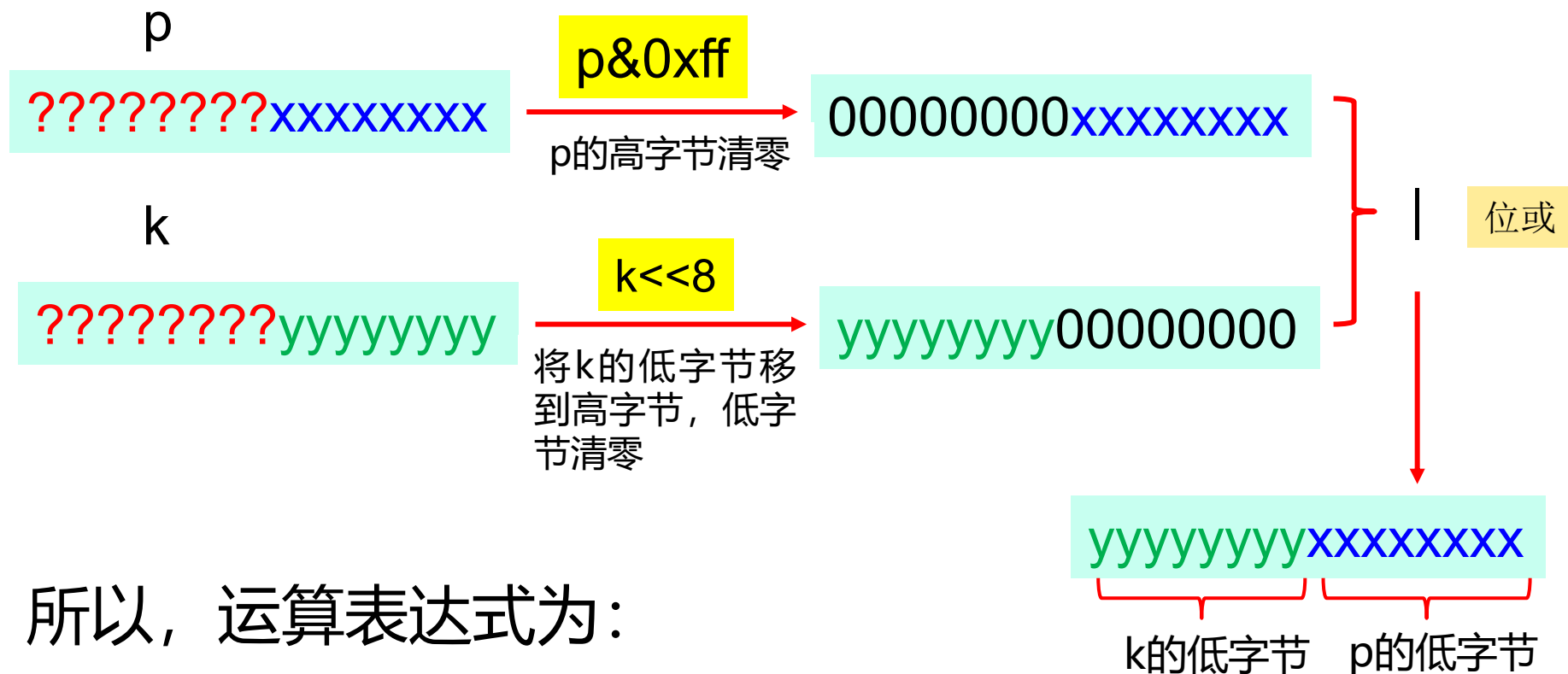
-9左移2位， $-9/4=-2$

移位运算符的例子

表达式	二进制表示	值	行为
a	00000000 00001011	11	
b	00000000 00001111	15	
-a	11111111 11110101	-11	对a进行负号运算 (求-a的补码)
a<<2	00000000 00101100	44	a左移2位(相当于乘4)
b>>3	00000000 00000001	1	b右移3位(相当于除8)
-a>>2	11111111 11111101	-3	-a右移2位, 符号位填1(也可能填0)

例2.22 写一个表达式，用短整数p的低字节作为结果的低字节，短整数k的低字节作为结果的高字节，拼成一个新的整数。

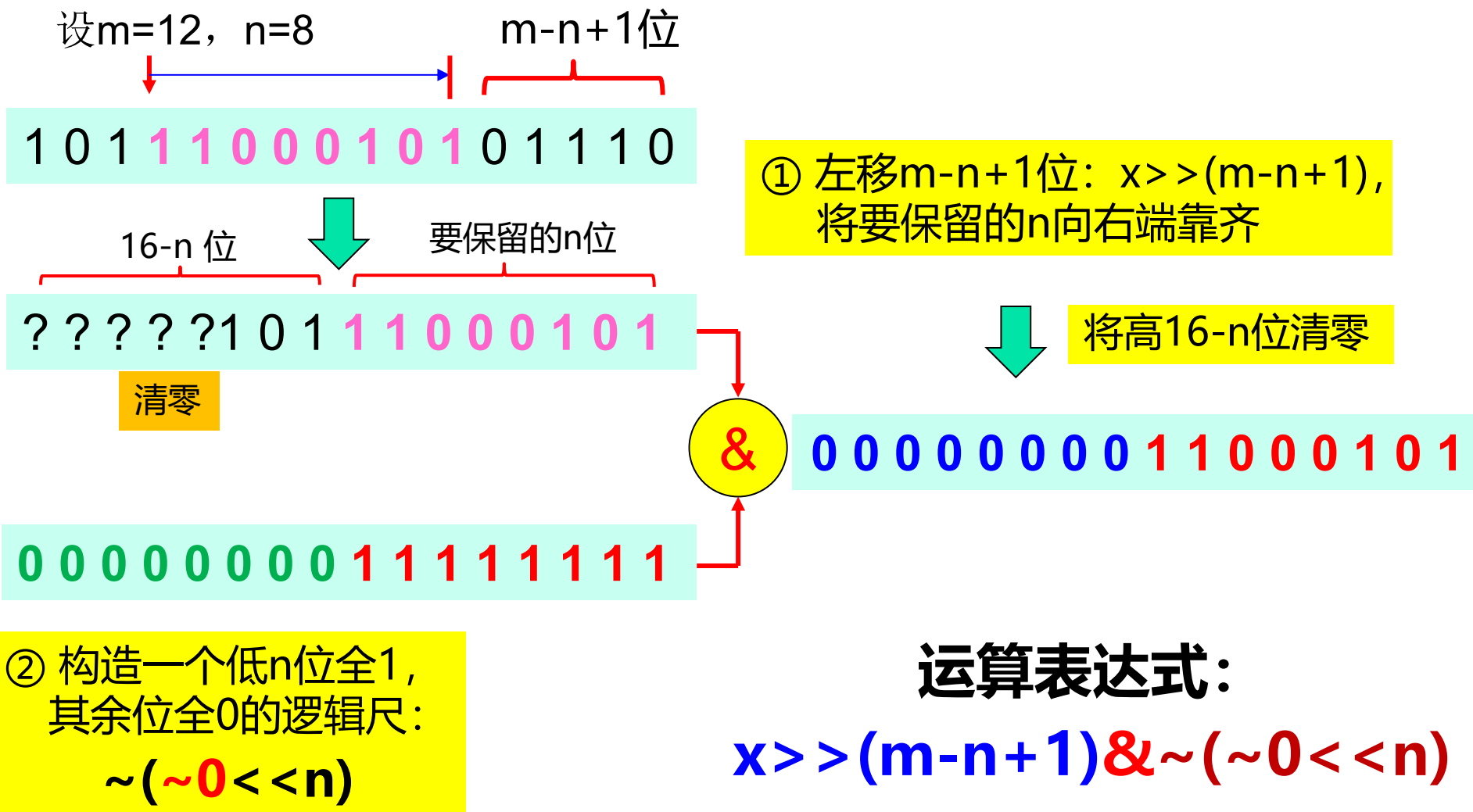
一个字节8位



所以，运算表达式为：

$(p \& 0xff) \mid (k \ll 8)$

例2.23 写一个表达式，取一个16位整数x从第m位开始向右的n位，并使其向右端靠齐。



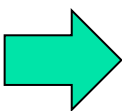
例2.24 可以把表示21世纪日期的日、月和年3个整数压缩成1个16位的整数。写一个表达式，实现压缩存储日、月和年。

分析：21世纪日期具有20yyymmdd的形式，其中，

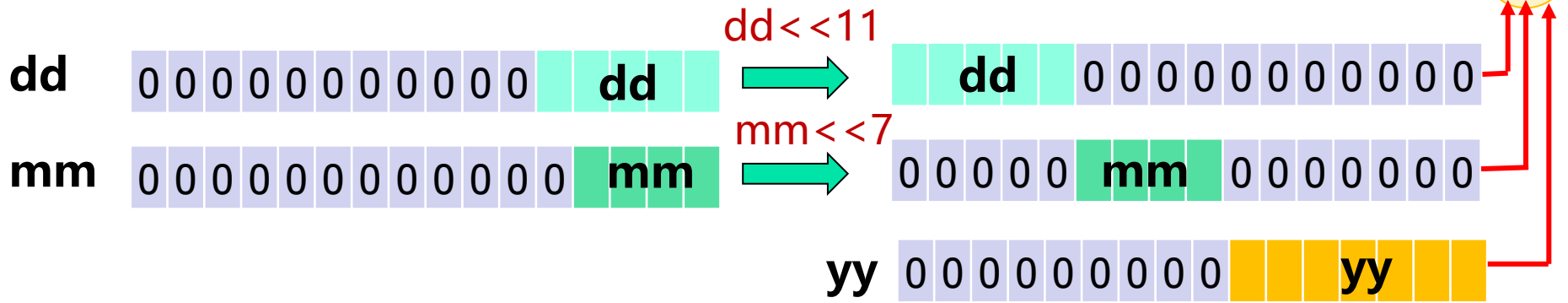
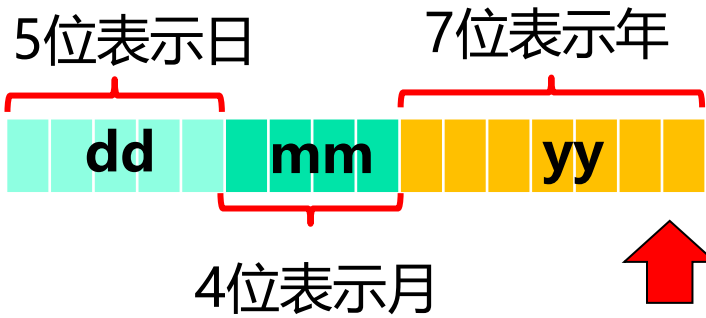
yy : 00~99

mm: 0~12

dd : 1~31

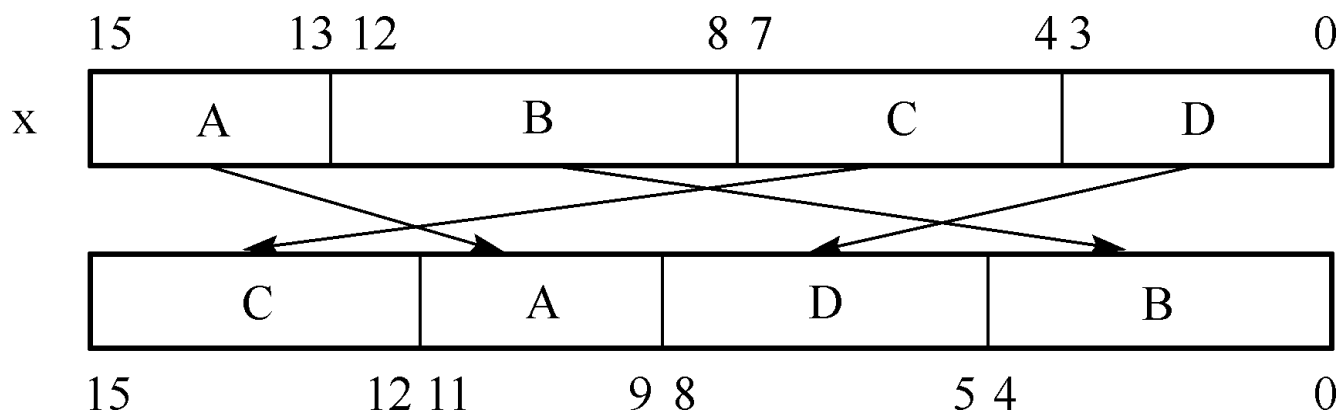


16位整数
(无符号)

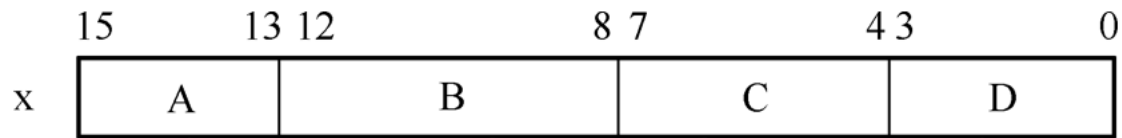


计算表达式： **day** << 11 | **month** << 7 | **year**

例2.25 简单加密示例。将一个短整型数 x 分成4个长度不等的部分：**A** (3b)、**B** (5b)、**C** (4b) 和**D** (4b)，然后将它们按照**CADB**的顺序重新拼凑在一起，实现对其加密的功能。写一个表达式，求其值为 x 的密文。



思路：先设计逻辑尺，利用逻辑尺提取各段内容；
然后将各段移位、合并（**或**在一起）



(1) 设计4个逻辑尺，分别提取 A、B、C、D 4个部分：

逻辑尺：

A: $x \ \& \ 0xE000;$ 1110000000000000

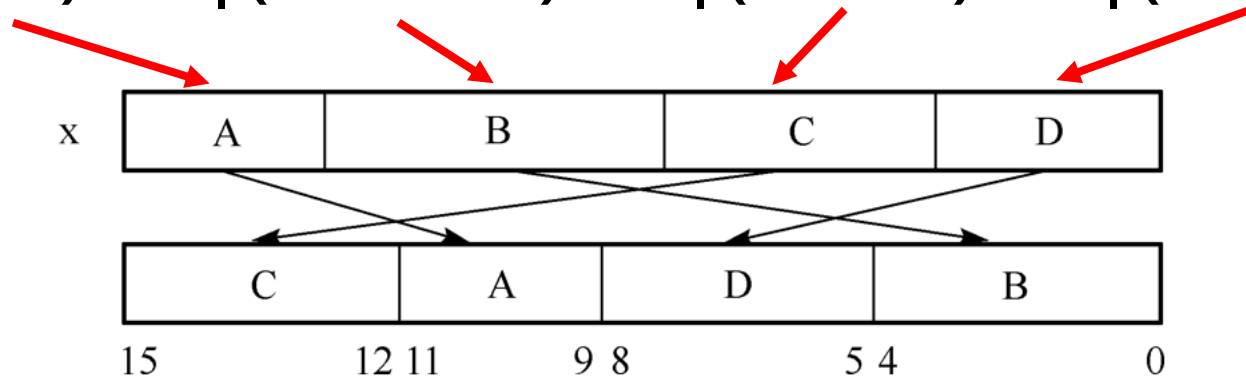
B: $x \ \& \ 0x1F00;$ 0001111100000000

C: $x \ \& \ 0x00F0;$ 0000000011110000

D: $x \ \& \ 0x000F;$ 0000000000001111

(2) 移位并合并组装，计算表达式为：

$(x \ \& \ 0xE000) \gg 4 \mid (x \ \& \ 0x1F00) \gg 8 \mid (x \ \& \ 0xF0) \ll 8 \mid (x \ \& \ 0xF) \ll 5$



例 2.26 将一个整数x以二进制方式输出。函数PrintBit的功能是输出int型数据的每一个二进制位。

例，设 $x=0xE1C4$

输出它的二进制形式：1111000111000100

设计思路：想办法读取x的每一位，如果该位是1则输出1，如果是0则输出0。

获取x每一位的方法：

首先设计一个最高位为1，其余各位为0的逻辑尺mask：

1000000000000000₂

然后用mask依次测试x的各位：

- ① 执行 $x \& \text{mask}$ ，提取**x的最高位**，输出该位的0/1值；
然后令 $x = x \ll 1$ ，即x左移1位，**原来的第二位变成现在的第一位**。
- ② 再次执行 $x \& \text{mask}$ ，提取当前最高位（**原始的第二位**），输出该位的0/1值；再令 $x = x \ll 1$ ，**原始的第三位变成现在的第一位**。
- ③ 再次执行 $x \& \text{mask}$ ，提取当前最高位（**原始的第三位**），输出该位的0/1值；再令 $x = x \ll 1$ ，**原始的第四位变成现在的第一位**。

.....

第16步完成后，x的16个二进制位就全部输出了。

用循环实现

```
#include<limits.h>
```

```
void bit_print(int x)
```

```
{
```

```
    int i;
```

```
    int n=sizeof(int) * 8;    /* 位数=字节数*8 */
```

```
    int mask = 1 << (n-1); /* 逻辑尺mask=100.....0 */
```

```
    for ( i=1; i<=n; ++i ) {
```

```
        putchar ( ! (x & mask) ? '0': '1'); /*只测试当前x的最高位*/
```

这一句也可以写成: `printf ("%d", x & mask ? 1: 0);`

```
        x<<=1;    /* x左移1位, 更新最高位 */
```

```
        if ( ! ( i % 8) && i<n ) putchar(' ');    /*控制格式, 每8位输出一个空格*/
```

```
    }
```

```
}
```

2.6 类型转换

C语言允许**双精度**、**单精度**、**整型**及**字符型**数据的**混合运算**,

如: **10 + '1' + 6.5**

在有混合类型的数据参加运算时, **先要对操作数进行数据类型转换, 类型一致后再进行计算。**

分四种情形:

- ◆ 整数提升
- ◆ 算术转换
- ◆ 赋值转换
- ◆ 强制转换

2.6.1 整数提升

当表达式中有**char**、**unsigned char**、**short**或**unsigned short** 这些“**短**”型数据时，则**首先将之转换成int或unsigned int**，然后参与后续计算。这种类型转换称为**整数提升**。

转换规则：如果原始类型的**表示范围小于int**，则转换成int，
否则转换成unsigned。

也就是，对**当前系统**中的int型（标准整型），如果其值域足够表示一个短型整数的值域，则将该短型整数提升为int型，否则提升成unsigned型。

- ◆ 类型提升的**根本目的**是扩充“短”数据的位数，使之达到系统**数据通路**的宽度（标准int型的位数）。
- ◆ 即使发生数据提升，也不改变原始变量的类型和值，而是生成一个内部中间结果，用这个中间结果参与后续运算。
- ◆ 类型提升的根本操作是用短数据的**符号位**填充扩展的位数。

例，设系统是16位。则char是8位，short、int是16位。

原始数据	原始数据的二进制表示	数据提升后的二进制表示（2字节）	数据提升后值的大小
char a=5;	00000101	0000000000000101	5 (int)
char b=-5;	1 1111011	11111111 1111011	-5 (int)
short x=5;	0000000000000101	0000000000000101	5 (unsigned)
short y=-5;	1111111111111011	1111111111111011	65529(unsigned)

这些是数据提升后中间值的结果，原始的a、b、x、y的值和类型没有变化

2.6.2 算术转换

对双目运算符的两个操作数，首先对每个操作数独立进行整数提升。如果提升之后两个操作数的类型还不相同，则进行**算术转换**。

算术转换的规则：

整数提升

char/short → int → unsigned → long → unsigned long
→ float → double → long double



简而言之： 值域较窄的类型向值域较宽的类型转换。

如：设有 **int i; float f;**

则 **i+f** 中，先把操作数 **i** 转换为**float**类型，然后进行计算，最后表达式 **i+f** 的类型也为**float**。

例 2.28 设有 `char c; long l; double d;` 写出表达式 `c*l+d` 所进行的类型转换。

解：

- ① `c`先被提升为`int`。 这一步是整数提升
- ② `c`再转换为`long`，和`l`相乘，结果为`long`型
- ③ 将`c*l`的结果（`long`型）转换为`double`，再和`d`相加，结果为`double`型。
- ④ 最后整个表达式的结果就为`double`型。

算术转换

例 求表达式 $-1 < 1U$ 的值。

解析：

因为 -1 为 `int`，而 $1U$ 为 `unsigned`，因此根据类型转换的规则， -1 被转换成 `unsigned`，大小为 **65535**。

(设系统中 `int` 为 2 个字节。 -1 的二进制补码表示为

1111111111111111_2

在被转型为无符号数后，该 16 位二进制表示的大小是 65535_{10} ，

亦即： -1 的内部字节编码不变，仅将该编码视为新类型的值)

所以最终表达式 $-1 < 1U$ 的值为 0（假）。

2.6.3 赋值转换

- ◆ 如果**赋值运算符**两侧的操作数类型不一致，则进行**赋值转换**。

赋值转换的规则：

将右操作数的值转换为左操作数类型要求的值，然后赋给

左操作数 —— 赋值表达式结果的类型最终由左操作数的类型决定。

(注：赋值转换不改变右操作数本来的值和类型，而是将其用左操作数类型要求的形式重新表示为一个中间结果，然后将中间结果赋给左操作数)

例如： `short s=5; double d=2.9;`

则， `s=d;` `/*把d的值转换为short型形式再赋给s， s的值为2*/`

`d=s;` `/*把s的值转换为double形式再赋给d， d的值为5.0*/`

2.6.4 强制类型转换

- 整数提升、算术转换、赋值转换都是**隐式转换**，系统自动完成；
- **强制类型转换是显式转换。**
- 强制类型转换的形式：**(类型名) 操作数**
- 强制类型转换不改变操作数本身的值和类型，而是将操作数的值转换成指定类型的另一个值（相当于产生一个中间结果）。

例如，int x, y;

(long)('a'-32) /* 'a' 被提升为int，减32后结果被强制转换为long型，大小是65*/

(float)x+y /*等价于((float)x)+y，结果是float类型*/

(double)x=10 /*错误， (double)x 是**表达式**，不能作为左值*/

(double)(x=10) /*正确， 将表达式(x=10)的值强制转换成double型*/

2.7 枚举类型

- ◆ **预定义数据类型**：char、int、float、double是C语言**内置的数据类型**，称为系统的**预定义数据类型**。
- ◆ **用户自定义数据类型**：基于基本数据类型，用户可以定义一些扩展类型，这些类型不是C语言本身所具有的，称为**用户自定义数据类型**。

枚举类型就是一种用户自定义类型。

注：枚举类型是一种类型的声明，其地位等同于int等类型。

例：用数字0~6表示星期日~星期六。

为了使用方便，可以“独立”定义符号常量：

```
#define SUN 0
```

```
#define MON 1
```

```
#define TUE 2
```

```
#define WED 3
```

```
#define THU 4
```

```
#define FRI 5
```

```
#define SAT 6
```

使用符号常量：

```
int day;
```

```
day = MON;
```



```
day = 1000;
```



这里，day只是一个普通的int型变量，赋值不受限制

➤ 可用但不好用

这些符号常量独立定义，书写繁琐，且没有形成一个有机的整体来表现“为表示星期几而专门设置的一些量”的这一层含义——体现不出彼此之间的相关性。

◆ 用**枚举类型**是来声明一个**符号常量的集合**来解决上述问题

如：以下定义了一个名字为**week**的**枚举类型**

```
enum week { SUN, MON, TUE, WED, THU, FRI, SAT };
```

关键字 ↑ 枚举名 ↑ 枚举常量

注：**enum** 是关键字；

week是枚举类型的名称，必须是合法标识符，代表后面的枚举常量集合；

该集合中有七个**符号常量**，而且只有这几个常量，表示enum week这种**枚举类型的“值域”**。

使用：**enum week day;**
day = MON;

- 定义枚举类型enum week的变量day
- 用枚举类型enum week中的枚举常量为枚举变量赋值

枚举常量是符号常量，其代表的值定义如下：

- (1) 在未指定值的缺省情况下，默认是枚举类型集合中**第一个枚举常量的值为0**，以后的值依次递增1。如：

```
enum week { SUN, MON, TUE, WED, THU, FRI, SAT };
```

枚举常量的值：**0 1 2 3 4 5 6**

- (2) 允许指定其中一个或多个枚举常量的值，而未指定值的枚举常量的值比前面的值大1。如：

```
enum sizes { SMALL, MEDIUM=10, BIG, TOO_BIG=20 };
```

枚举常量的值：**0 10 11 20**

2.7.1 枚举类型定义的一般形式

enum [**枚举名**] {**标识符**[= **常量表达式**], **标识符**[= **常量表达式**], ...};

其中，**枚举名**是该枚举类型的名字；

标识符是集合中枚举量的名字，称为**枚举常量**；

常量表达式用于给这些枚举常量指定初值；

- ◆ 枚举类型是一种**用户自定义类型**，必须**先定义才能使用**。
- ◆ 枚举类型适用于取值为整数、取值范围有限、枚举量个数不多的应用场合。

◆ 定义枚举类型时，enum 后面可以不出现枚举名。

如：enum { WIN, LOSE, TIE, ERROR};

◆ 枚举变量只能用其类型中已有的枚举常量赋值，不能用数字或其它数据为其简单赋值。

设：enum week { SUN, MON, TUE, WED, THU, FRI, SAT };

enum week day;

day = MON; ✓

day = 1; ✗

用枚举类型声明枚举变量时，关键字enum不能少。

2.7.2 用枚举类型定义符号常量

以前用#define定义符号常量：

```
#define WIN 0
#define LOSE 1
#define TIE 2
#define ERROR -1
```

现在可用枚举类型定义来代替：

```
enum { WIN, LOSE, TIE, ERROR=-1};
```

注：枚举类型定义中，花括号中的每个名字都是一个符号常量。
枚举类型的这种定义形式体现出这些**符号常量间的内在相关性**。

2.7.3 声明枚举变量

用**enum**声明的枚举类型声明的该类型的变量，称为**枚举变量**。

有两种定义形式：

(1) 在定义枚举类型的同时声明枚举变量

如：**enum color { RED, GREEN, BLUE} c1,c2;**

(2) 先定义枚举类型，再声明枚举变量

如：**enum color { RED, GREEN, BLUE} ;**

enum color c1, c2;

枚举类型

用枚举类型声明枚举变量时，关键字**enum**不能少。

■ 使用枚举变量

- (1) 操作时，**枚举变量的值被解释成int型整数**，因此它可以出现在整数允许出现的任何地方。但**枚举变量的取值仅限于枚举集合中列举出来的值**，不在该范围内的值将出错。
- (2) 程序中**为枚举变量赋值只能用其类型定义的符号常量集中的符号常量**，不能用对应的整数值。
- (3) 但**枚举变量的输入和输出都是整数**，而不是符号常量名，且整数的值应在类型定义的符号常量集以内。

如： `enum color { RED, GREEN, BLUE} c1, c2;`

`c1=BLUE;` `/* 不能用 c1=2; */`

`printf("%d", c1);` `/* 输出2，而不是BLUE */`

`scanf("%d", &c2);` `/* 可输入0、1、2，但不能输入RED、
GREEN、BLUE等*/`

◆ 下面的语句是错误的：

```
enum color { RED, GREEN, BLUE} c1, c2;
```

```
c1=3;  /* 不能用整数，且3超出了枚举类型enum color的取值范围 */
```

```
printf("%s", GREEN );  /* GREEN是符号常量名，不是字符串 */
```

◆ 根据枚举变量的值输出文本信息的正确方法：

```
if(c1==RED) printf("Red" );
```

```
else if(c1==GREEN) printf("Green" );
```

```
else if(c1==BLUE) printf("Blue" );
```

例2.22 定义一个描述星期的枚举类型，输入表示星期的数字1~7，7代表星期日。然后输出对应的英文星期名。**(课下编程)**

分析：定义一个以星期英文名为元素的**字符串数组**，**以枚举值作为数组下标**来索引该字符串数组中的星期名，用printf输出。

```
#include <stdio.h>
```

```
enum week { MON=1, TUE, WED, THU, FRI, SAT, SUN} ;
```

值: 1 2 3 4 5 6 7

```
int main(void)
```

```
{
```

字符串数组



```
enum week day;
```

```
char * weekName[ ] = { "Input error!", "Monday", "Tuesday",  
                        "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"};
```

```
printf("请输入1-7的星期数字\n");
```

```
while(scanf("%d", &day) ==1) /*以整数的方式输入星期数字 */
```

```
if(day>=MON && day <= SUN)
```

```
    printf("%s\n", weekName[day] );
```

```
    else printf("%s\n", weekName[0] );
```

```
return 0;
```

```
}
```

2.8 C99新增数据类型

C99标准增加了支持64位的整数类型long long，还增加了布尔类型和复数类型。CodeBlocks编译器能够较好地支持C99标准，本节中的例子程序均由CodeBlocks运行通过。

2.8.1 long long类型

long long类型的长度至少应为64位。和其他整型类型一样，有带符号和无符号两种：

- **signed long long**：数值范围为 $-2^{63} \sim 2^{63}-1$
- **unsigned long long**：数值范围为 $0 \sim 2^{64}-1$ 。
- long long类型的常量用后缀“LL”或“ll”表示。

注：在所有整型类型中，类型long long的值域最宽，因此，C99标准的自动转换所遵循的转换方向为：

...→unsigned long→**long long**→**unsigned long long**→float→...

例2.23 输出斐波那契（FIBONACCI）数列的前60个数。

分析：

- FIBONACCI数列：1, 1, 2, 3, 5, 8,。即从第三个数开始，每个数是它前面两数的和。
- 由于FIBONACCI数列从大约**第45个数**起就超过了long类型的范围，为避免溢出，应将结果变量说明为long long类型，从输出结果可以看到，它的长度为8字节。

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    long long f1 = 1, f2 = 1;
```

```
    int i;
```

```
    printf("sizeof(long long )=%d\n", sizeof(f1));
```

```
    for (i=1; i<=30; i++)
```

```
    {
```

```
        printf("f(%d)=%I64d\nf(%d)=%I64d\n", (2*i-1), f1, i*2, f2);
```

```
        f1 = f1 + f2;
```

```
        f2 = f2 + f1;
```

```
    }
```

```
    return 0;
```

```
}
```

每次计算和输出两个数，
循环30次，共输出60个数

2.8.2 布尔类型

- C语言中可以用任何整数类型表示布尔值，0表示假，所有非0表示真。布尔表达式为假时值为0，为真时值为1。例如， $i=(a<b)$ ，在a小于b时对整型变量i赋值1；在a不小于b时对变量i赋值0。
- **C99标准引入了真正的布尔类型 `_Bool`**，`_Bool`类型长度为1，只能保存数值0和1（分别表示“**false**”与“**true**”）。虽然也可以使用其他整数类型表示布尔值，但如果C语言实现支持C99标准，那么使用`_Bool`类型更清晰。
- 另外，C99标准为了让C和C++兼容，增加了一个头文件**`stdbool.h`**。里面定义宏名`bool`为`_Bool`的同义词，定义`false`与`true`分别为0和1。因此，只要在源文件中包含`stdbool.h`这个头文件，就可以在C语言中里像C++那样使用`bool`定义布尔类型变量，通过`true`和`false`对布尔变量进行赋值。
- 将任意非零值赋值给`_Bool`类型的变量，都会先转换为1，表示真。将零值赋值给`_Bool`类型的变量，结果为0，表示假。

例 捕鱼和分鱼问题

A、B、C、D、E五人在某天夜里合伙捕鱼，到第二天凌晨时都疲惫不堪，于是各自找地方睡觉。A第一个醒来，他将鱼分为五份，把多余的一条扔掉，拿走自己的一份。B第二个醒来，也将鱼分为五份，把多余的一条扔掉，拿走自己的一份。C、D、E依次醒来，也按同样的方法拿鱼。问他们合伙捕了多少条鱼？

分析：总共将所有的鱼进行了5次平均分配，每次分配的策略是相同的，即扔掉一条后剩下的可均分为五份，然后拿走自己的一份，余下其他的四份。

假定鱼的总数为 X ，则 X 可以按照题目的要求进行五次分配： $X-1$ 后可被5整除，余下的鱼为 $4*(X-1)/5$ 。若 X 满足上述要求，则 X 就是题目的解。

```
#include <stdio.h>
#include <stdbool.h>
int main(void)
{
    int n, i, x;
    bool flag = true;           /*flag: 控制标记*/
    for (n=6; flag; n++)       /*采用试探的方法 (模拟法) , 令试探值n逐步加大*/
    {
        for (x=n, i=1; flag && i<=5; i++)/*判断是否可以按照题目要求进行5次分配*/
        {
            if ((x-1)%5 == 0)
                x = 4 * (x - 1) / 5;
            else
                flag = false;    /*若不能分配则设置标记flag=0, 退出当前测试*/
        }
        if(flag)
            break;               /*若分配过程正常结束则找到结果退出整个试探过程*/
        else
            flag=true;         /*否则继续试探下一个数*/
    }
    printf("Total number of fish caught=%d\n", n);

    return 0;
}
```

2.8.3 复数类型

- C99标准新增了2个复数类型关键字：

_Complex 和 **_Imaginary**

其中，_Imaginary表示只有一个实数组成的纯虚数类型（即bi）。由于复数的实部和虚部均为实数，在说明复数类型时，需用**浮点类型名**进一步指定对应实数类型，因此复数或纯虚数各有以下3种类型：

- float _Complex、double _Complex、long double _Complex
- float _Imaginary、double _Imaginary和long double _Imaginary

- 复数类型 `_Complex` 表示为对应实数类型的二元数组，第1个元素表示复数的**实数部分**，第2个元素表示复数的**虚数部分**。而复数类型 `_Imaginary` 是实数部分总为0的纯虚数类型。
- C99标准还引入了头文件 `complex.h`，其中定义 `complex` 宏为关键字 `_Complex` 的同义词，定义 `imaginary` 宏为关键字 `_Imaginary` 的同义词，定义 `_Complex_I` (或 `I`) 为复数类型的**虚数常量**表达式，取值为虚数单位 i 。因此，**复数型常量**用包含了虚数常量 `_Complex_I` (或 `I`) 的**浮点型常量表达式**来表示，如 $1.0+2.0*I$ ， $1.0+2.0*I$ ， $4.5*I$ 等。
- 复数的运算由相应的库函数提供支持。在C99标准中，复数的库函数很多，如三角函数、双曲函数、指数函数和其他函数等，这些函数的原型在头文件 `complex.h` 中说明。

复数类型转换规则

- (1) 一个复数类型转换成另一个复数类型时，实数和虚数浮点数成员按照浮点类型转换规则分别转换。
- (2) 从Complex类型转换为_Imaginary类型时，放弃实数部分；从_Imaginary类型转换为_Complex类型时，将实数部分设置为0。
- (3) 将一个实数（整数或浮点数）转换成复数类型时，该实数转换成复数值的实数部分，而虚数部分为0。
- (4) 将一个复数类型转换成实数（整数或浮点数）时，虚数部分放弃。

例2.25 求两个复数的乘积。

```
#include <stdio.h>
#include <complex.h>
int main(void)
{
    double _Complex a, b, c;
    printf("The size of a is %d\n", sizeof(a));
    a = 2 + 3 * I;
    b = 4 + 5 * I;
    c = a * b;
    printf("c=%f+%f*I\n", creal(c), cimag(c));
    return 0;
}
```

说明：程序中定义了3个双精度的复数a、b和c，第一条printf函数调用语句输出复数a所需的内存大小，它为**double的2倍**。

接着，给复数变量a和b分别赋一个复数常量，复数a和b的乘积赋值给复数变量c。函数**creal**和**cimag**分别为取一个复数的实部和虚部。

```
The size of a is 16
c=-7.000000+22.000000*I
```

本章小结

- C语言的字符集。
- 关键字、标识符。
- C语言的基本数据类型：字符型、整型、浮点型。
 - 字符串
 - 在C99标准中增加了布尔类型、长长整型、复数类型等。
- 常量和变量的概念、定义和使用
- C语言的运算：算术运算、关系运算、逻辑运算、赋值运算、复合赋值运算、自增/自减运算、条件运算、逗号运算、sizeof和位运算；**运算符的优先级和结合性规则。**
- 表达式和表达式的值与类型。
- 类型转换问题：自动类型转换（整数提升、算数转换、赋值转换）和强制类型转换。